

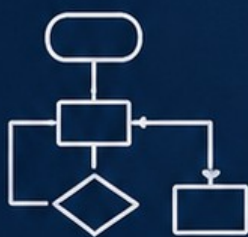
INTRODUÇÃO AO PORTUGOL

LÓGICA DE PROGRAMAÇÃO PARA INICIANTES

Aprenda a programar do zero com exemplos práticos e exercícios passo a passo.



ALGORITMOS



LÓGICA



ESTRUTURAS



REPETIÇÃO

```
programa
inicio
  inteiro n
  escreva("Digite um número:")
  leia(n)
  se (n % 2 = 0) entao
    escreva("Par")
  senao
    escreva("Ímpar")
  fimse
finalgoritmo
```



PROGRAMAS



PENSE LÓGICO.
PROGrame SOLUÇÕES.



DO CONCEITO
À PRÁTICA.

UM GUIA
SIMPLES, PRÁTICO
E ACESSÍVEL
PARA COMEÇAR
A PROGRAMAR!



Anhanguera

**PROJETO DE EXTENSÃO I
ANÁLISE E DESENVOLVIMENTO DE SISTEMAS**

Maicon Bruno Araujo de Melo
Matrícula: 2024177810

Polo Goiânia
2026

1. ÍNDICE

2. CAPÍTULO 1 – Introdução a programação e pensamento computacional
3. CAPÍTULO 2 – Introdução ao portugol
4. CAPÍTULO 3 – Variaveis e tipos de dados
5. CAPÍTULO 4 – Variáveis e Tipos de Dados
6. CAPÍTULO 5 – Operadores
7. CAPÍTULO 6 – Estruturas Condicionais
8. CAPÍTULO 7 – Estruturas de Repetição
9. CAPÍTULO 8 – Vetores e Matrizes
10. CAPÍTULO 9 – Procedimentos e Funções
11. CAPÍTULO 10 – Boas Práticas
12. Conclusão
13. Referências

CAPÍTULO 1 – INTRODUÇÃO À PROGRAMAÇÃO E AO PENSAMENTO COMPUTACIONAL

1.1 O que é Programação?

A programação é o processo de criar instruções que serão executadas por um computador para resolver problemas ou realizar tarefas específicas. Essas instruções são organizadas de forma lógica e ordenada, formando o que chamamos de algoritmo.

No cotidiano, utilizamos algoritmos constantemente, mesmo sem perceber. Uma receita culinária, um manual de montagem ou as instruções para utilizar um caixa eletrônico são exemplos de algoritmos presentes em nossa rotina.

Quando programamos, estamos criando algoritmos capazes de serem compreendidos por máquinas.

Exemplo:

Para calcular a média de duas notas, podemos seguir os seguintes passos:

1. Receber a primeira nota.
2. Receber a segunda nota.
3. Somar as duas notas.
4. Dividir o resultado por dois.
5. Exibir a média calculada.

Esses passos formam um algoritmo simples.

1.2 O que é um Computador?

Um computador é um equipamento eletrônico capaz de receber dados, processá-los e produzir resultados.

Seu funcionamento pode ser representado por três etapas fundamentais:

ENTRADA → PROCESSAMENTO → SAÍDA

Exemplos:

Entrada:

- Teclado
- Mouse
- Scanner

Processamento:

- CPU (Unidade Central de Processamento)

Saída:

- Monitor

- Impressora
- Alto-falantes

Todo programa segue esse modelo básico.

O usuário fornece informações, o computador processa essas informações e apresenta um resultado.

1.3 Hardware e Software

Para compreender a programação é importante diferenciar hardware de software.

Hardware

Hardware é toda a parte física do computador.

Exemplos:

- Processador
- Memória RAM
- Placa-mãe
- HD ou SSD
- Monitor
- Teclado

Software

Software é a parte lógica do sistema.

Exemplos:

- Sistemas Operacionais
- Jogos
- Navegadores
- Aplicativos
- Programas desenvolvidos pelo programador

O hardware executa as instruções definidas pelo software.

1.4 O que é um Algoritmo?

Um algoritmo é uma sequência finita de passos organizados para solucionar um problema.

Para que um algoritmo seja considerado correto, ele deve possuir:

Clareza

As instruções devem ser objetivas e sem ambiguidades.

Ordem

Os passos devem seguir uma sequência lógica.

Finitude

O algoritmo deve terminar após executar suas etapas.

Eficiência

Deve resolver o problema utilizando recursos adequadamente.

Exemplo de algoritmo para escovar os dentes:

1. Pegar a escova.
2. Colocar pasta dental.
3. Escovar os dentes.
4. Enxaguar a boca.
5. Guardar a escova.

Caso alguma etapa seja removida ou executada fora de ordem, o resultado esperado pode não ser alcançado.

1.5 Pensamento Computacional

O pensamento computacional é uma forma de resolver problemas utilizando princípios da Ciência da Computação.

Ele é composto por quatro pilares fundamentais.

Decomposição

Consiste em dividir um problema complexo em problemas menores.

Exemplo:

Sistema escolar:

- Cadastro de alunos
- Cadastro de professores
- Lançamento de notas
- Emissão de boletins

Cada módulo resolve uma parte do problema geral.

Reconhecimento de Padrões

Consiste em identificar situações semelhantes já resolvidas anteriormente.

Exemplo:

Se já sabemos calcular a média de um aluno, podemos aplicar a mesma lógica para vários alunos.

Abstração

Consiste em focar apenas nas informações relevantes.

Exemplo:

Para calcular a média de um estudante, não precisamos saber sua altura ou cor favorita.

Algoritmos

Após compreender o problema, criamos uma sequência de passos para resolvê-lo.

1.6 Representação de Algoritmos

Os algoritmos podem ser representados de diferentes maneiras.

Linguagem Natural

Utiliza a linguagem comum do ser humano.

Exemplo:

"Leia duas notas e calcule a média."

Fluxograma

Utiliza símbolos gráficos para representar o fluxo do algoritmo.

Exemplo simplificado:

```

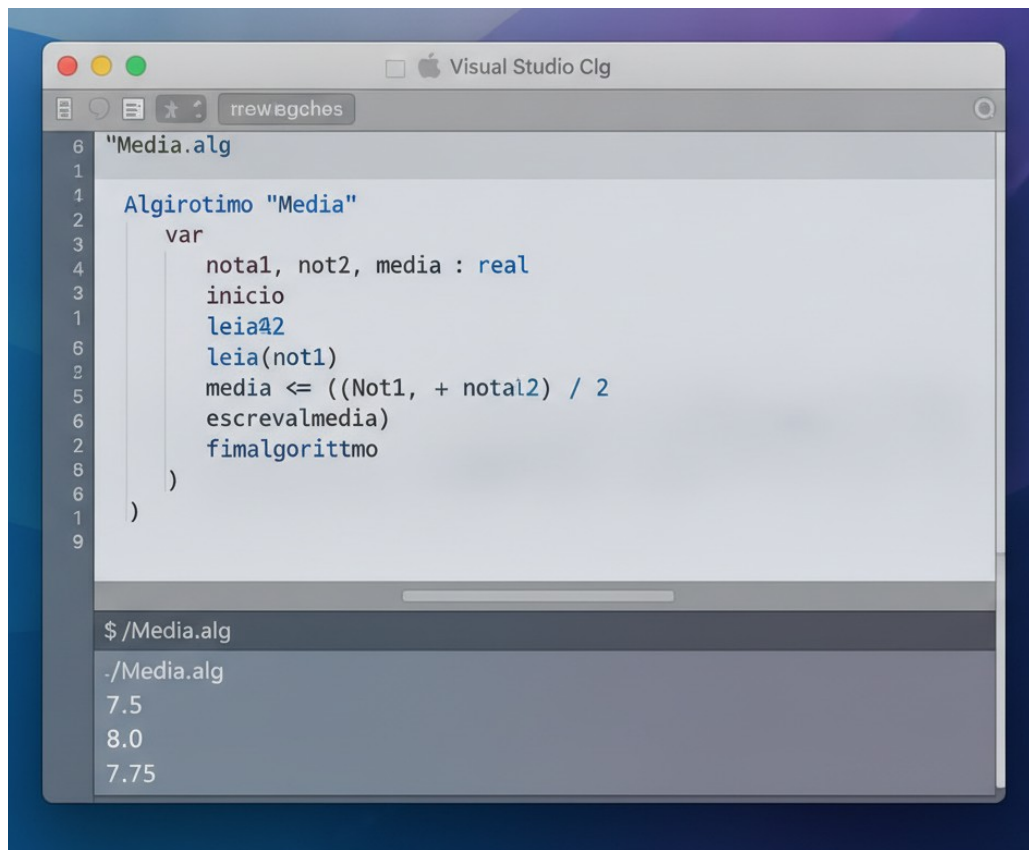
Início
↓
Ler Nota 1
↓
Ler Nota 2
↓
Calcular Média
↓
Exibir Média
↓
Fim

```

Portugol

É uma linguagem educacional utilizada para ensinar lógica de programação.

Exemplo:



O Portugol permite que o estudante se concentre na lógica sem se preocupar inicialmente com detalhes específicos de linguagens profissionais.

1.7 Como o Computador Entende um Programa?

Os seres humanos escrevem programas utilizando linguagens de programação como Portugol, Python, Java ou C#.

Entretanto, o computador não compreende palavras, frases ou comandos escritos nessas linguagens.

Internamente, o computador entende apenas sinais elétricos representados por dois valores:

0 = Desligado

1 = Ligado

Essa linguagem é chamada de **Código de Máquina** ou **Linguagem Binária**.

Exemplo:

10110100

01010110

11001101

Programar diretamente dessa forma seria extremamente difícil.

Por esse motivo foram criadas linguagens de programação mais próximas da linguagem humana.

O que é um Compilador?

Um compilador é um programa responsável por traduzir o código escrito pelo programador para uma linguagem que o computador consiga executar.

Representação:

Programador

↓

Código Fonte

↓

Compilador

↓

Código de Máquina

↓

Computador

Exemplo:

Código escrito pelo programador:

```
escreva("Olá Mundo")
```

Após a compilação:

```
101010010101
```

```
010101001001
```

```
101001010101
```

O usuário não vê essa conversão, mas ela ocorre internamente.

O que é um Interpretador?

Algumas linguagens utilizam interpretadores em vez de compiladores.

A principal diferença está na forma de execução.

Compilador

Traduz todo o programa antes da execução.

Código

↓

Compilar



Executar

Exemplos:

- C
- C++
- Pascal

Interpretador

Traduz e executa cada instrução conforme o programa está sendo executado.

Linha 1 → Executa

Linha 2 → Executa

Linha 3 → Executa

Exemplos:

- Python
- JavaScript
- PHP

E o Portugol?

O Portugol é uma linguagem educacional.

Dependendo do ambiente utilizado, como Visualg ou Portugol Studio, existe um interpretador que lê os comandos escritos pelo estudante e simula sua execução.

Exemplo:

```
algoritmo "Ola"
```

```
inicio
```

```
escreval("Olá Mundo")
```

```
fimalgoritmo
```

Ao pressionar executar, o interpretador do Portugol analisa cada instrução e apresenta o resultado na tela.

Por que isso é importante?

Entender compiladores e interpretadores ajuda o estudante a compreender que:

- O computador não entende diretamente a linguagem humana.
- Existe um processo de tradução.
- Erros de programação são identificados durante essa tradução.
- Toda linguagem de programação precisa ser convertida para algo que o processador consiga executar.

Esse conhecimento servirá como base para o aprendizado de linguagens profissionais nos próximos níveis de estudo.

1.8 Características de um Bom Programador

Um bom programador não é aquele que apenas conhece comandos.

As principais características de um desenvolvedor são:

- Capacidade de resolver problemas.
- Organização.
- Pensamento lógico.
- Atenção aos detalhes.
- Disposição para aprender continuamente.
- Capacidade de testar e corrigir erros.

Programação é muito mais sobre raciocínio do que sobre memorização.

Resumo do Capítulo

Neste capítulo aprendemos:

- O conceito de programação.
- O funcionamento básico de um computador.
- A diferença entre hardware e software.
- O que é um algoritmo.
- Os pilares do pensamento computacional.
- Formas de representar algoritmos.
- A importância da lógica para o desenvolvimento de sistemas.

Esses conceitos servirão como base para todos os capítulos seguintes.

Exercícios Propostos

1. Explique com suas palavras o que é programação.
2. Diferencie hardware e software.
3. Cite três exemplos de dispositivos de entrada e três de saída.
4. O que é um algoritmo?
5. Quais são os quatro pilares do pensamento computacional?
6. Explique o conceito de abstração.

7. Por que o Portugol é utilizado no ensino de programação?
8. Elabore um algoritmo em linguagem natural para preparar um café.
9. Elabore um algoritmo em linguagem natural para sacar dinheiro em um caixa eletrônico.
10. Pesquise e descreva outras áreas em que a programação é utilizada atualmente.

CAPÍTULO 2 – INTRODUÇÃO AO PORTUGOL

2.1 O que é Portugol?

O Portugol é uma linguagem de programação educacional criada para facilitar o aprendizado da lógica de programação. Seu objetivo é permitir que estudantes desenvolvam algoritmos utilizando comandos escritos em português, tornando o processo de aprendizagem mais intuitivo.

Embora não seja utilizado diretamente no mercado de trabalho para desenvolvimento de sistemas comerciais, o Portugol desempenha um papel fundamental na formação do raciocínio lógico do programador.

Ao dominar Portugol, o estudante estará preparado para aprender linguagens profissionais como Python, Java, C#, JavaScript e C++.

2.2 Estrutura Básica de um Algoritmo

Todo algoritmo em Portugol possui uma estrutura padrão.

Exemplo:

algoritmo "PrimeiroPrograma"
inicio
escreval("Olá Mundo!")
fimalgoritmo

Observe os elementos principais:

algoritmo

Define o início do programa.

algoritmo "PrimeiroPrograma"

O nome deve representar o objetivo do algoritmo.

inicio

Indica o início da execução das instruções.

inicio

fimalgoritmo

Indica o encerramento do programa.

fimalgoritmo

Tudo que estiver entre inicio e fimalgoritmo será executado.

2.3 Identificadores

Identificadores são os nomes utilizados para variáveis, procedimentos e funções.

Exemplos válidos:

idade

nome_aluno

mediaFinal

valor_total

Exemplos inválidos:

1idade

nome-aluno

valor total

@usuario

Boas práticas:

- Utilizar nomes descritivos.
- Evitar abreviações excessivas.
- Utilizar letras minúsculas.
- Separar palavras com underline (_).

Exemplo:

quantidade_produtos

media_final

salario_funcionario

2.4 Comentários

Comentários são utilizados para documentar o código.

Eles não são executados pelo computador.

Exemplo:

// Calcula a média do aluno

```
media <- (nota1 + nota2) / 2
```

Comentários ajudam outros programadores a compreenderem o funcionamento do algoritmo.

Devem ser utilizados apenas quando realmente agregam informação.

Comentários excessivos podem dificultar a leitura.

2.5 Comandos de Saída

Os comandos de saída permitem exibir informações ao usuário.

escreva

Exibe informações sem quebrar linha.

Exemplo:

```
escreva("Digite seu nome: ")
```

Resultado:

Digite seu nome:

O cursor permanece na mesma linha.

escreval

Exibe informações e realiza uma quebra de linha.

Exemplo:

```
escreval("Sistema iniciado")
```

Resultado:

Sistema iniciado

O cursor será movido para a próxima linha.

2.6 Comandos de Entrada

O comando de entrada permite que o usuário informe dados.

leia

Exemplo:

```
leia(nome)
```

O programa ficará aguardando a digitação do usuário.

Fluxo:

Programa → Solicita dado

Usuário → Digita informação

Programa → Armazena informação

2.7 Primeiro Programa Interativo

Vamos criar um programa simples para cumprimentar o usuário.

algoritmo "BoasVindas"

var

nome : caractere

inicio

escreva("Digite seu nome: ")

leia(nome)

escreval("Olá ", nome)

fimalgoritmo

Execução:

Digite seu nome: João

Olá João

Observe que o valor digitado pelo usuário foi armazenado na variável e utilizado posteriormente.

2.8 Fluxo de Execução de um Programa

Todo programa segue uma sequência lógica de execução.

Exemplo:

algoritmo "Exemplo"

inicio

escreval("Passo 1")

escreval("Passo 2")

escreval("Passo 3")

fimalgoritmo

Saída:

Passo 1

Passo 2

Passo 3

O computador executa as instruções de cima para baixo.

Essa característica é chamada de fluxo sequencial.

2.9 Erros Comuns de Iniciantes

Esquecer o inicio

Incorreto:

```
algoritmo "Teste"
```

```
escreval("Olá")
```

Correto:

```
algoritmo "Teste"
```

```
inicio
```

```
escreval("Olá")
```

```
fimalgoritmo
```

Esquecer o (fimalgoritmo)

Todo programa deve possuir encerramento.

Utilizar variáveis não declaradas

Incorreto:

```
leia(nome)
```

Sem declaração prévia.

Correto:

```
var
```

```
nome : caractere
```

Utilizar nomes inadequados

Evite:

```
a
```

```
b
```

```
x
```

```
y
```

Prefira:

nome
idade
salario
media

2.10 Exemplo Completo

O algoritmo abaixo lê o nome e a idade de uma pessoa.

```
1  programa {
2      funcao inicio() {
3          // Declaração de variáveis
4          cadeia nome
5          inteiro idade = 0
6
7          // Entrada de dados
8          escreva("Digite seu nome: ")
9          leia(nome)
10
11         escreva("Digite sua idade: ")
12         leia(idade)
13
14         // Saída de dados
15         escreva("\n") // O Portugol Studio usa \n para pular linha, ou apenas escreva() vazio
16         escreva("Dados cadastrados:\n")
17         escreva("Nome: ", nome, "\n")
18         escreva("Idade: ", idade, "\n")
19     }
20 }
```

Resultado:

Dados cadastrados:

Nome: ANA

Idade: 18

Programa finalizado.

Resumo do Capítulo

Neste capítulo aprendemos:

- O que é Portugol.
- Estrutura básica de um algoritmo.
- Identificadores.
- Comentários.
- Comandos de entrada e saída.
- Fluxo de execução.
- Erros comuns de iniciantes.

Esses conceitos serão utilizados em todos os programas desenvolvidos ao longo do livro.

Exercícios Propostos

1. Crie um algoritmo que exiba a mensagem "Bem-vindo à Programação".
2. Crie um algoritmo que solicite o nome do usuário e exiba uma saudação.
3. Crie um algoritmo que solicite nome e cidade do usuário e exiba os dados.
4. Crie um algoritmo que mostre três mensagens em linhas diferentes.
5. Explique a diferença entre escreva e escreval.
6. Explique a função do comando leia.
7. Cite três exemplos de identificadores válidos.
8. Cite três exemplos de identificadores inválidos.
9. Escreva um algoritmo que exiba seu nome, idade e curso.
10. Explique por que comentários são importantes em um programa.

CAPÍTULO 3 – VARIÁVEIS E TIPOS DE DADOS

3.1 O que são Variáveis?

Durante a execução de um programa, frequentemente precisamos armazenar informações para utilizá-las posteriormente. Essas informações são guardadas em estruturas chamadas variáveis.

Uma variável pode ser entendida como uma caixa identificada por um nome, onde armazenamos um determinado valor.

Exemplo:

Nome da Caixa: idade

Valor Armazenado: 18

Ao longo da execução do programa, o valor armazenado pode ser alterado diversas vezes.

Por isso recebe o nome de variável.

3.2 Variáveis na Memória

Quando declaramos uma variável, o computador reserva um espaço na memória RAM para armazenar os dados.

Representação simplificada:

+-----+

| Nome: idade |

| Valor: 18 |

+-----+

+-----+

| Nome: salario |

| Valor: 2500.50 |

+-----+

Sempre que o programa precisar utilizar essas informações, ele acessará os locais correspondentes na memória.

3.3 Declaração de Variáveis

Antes de utilizar uma variável, devemos declará-la na seção var.

Sintaxe:

var

nome_variavel : tipo

Exemplo:

var

idade : inteiro

Nesse caso, a variável idade poderá armazenar números inteiros.

3.4 Tipos de Dados

Cada variável deve possuir um tipo específico.

O tipo define quais valores poderão ser armazenados.

Os tipos mais utilizados em Portugol são:

Inteiro

Armazena números sem casas decimais.

Exemplos:

10

-5

0

250

Declaração:

var

idade : inteiro

Real

Armazena números com casas decimais.

Exemplos:

1.5

7.25

1500.99

Declaração:

var

salario : real

Caractere

Armazena textos.

Exemplos:

"Maria"

"Programação"

"ADS"

Declaração:

var

nome : caractere

Lógico

Armazena apenas dois estados:

- VERDADEIRO
- FALSO

Declaração:

var

aprovado : logico

3.5 Atribuição de Valores

A atribuição é realizada utilizando o operador:

`<-`

Esse operador indica que um valor será armazenado em uma variável.

Exemplo:

`idade <- 20`

Após a execução:

`idade = 20`

Outro exemplo:

`nome <- "Carlos"`

Resultado:

`nome = Carlos`

3.6 Alterando Valores

O conteúdo de uma variável pode ser modificado durante a execução do programa.

Exemplo:

`idade <- 18`

`idade <- 19`

`idade <- 20`

Ao final:

`idade = 20`

O valor anterior é substituído pelo novo valor.

3.7 Entrada de Dados em Variáveis

A maneira mais comum de preencher variáveis é utilizando o comando `leia`.

Exemplo:

`var`

`nome : caractere`

`inicio`

`leia(nome)`

fimalgoritmo

O valor digitado pelo usuário será armazenado dentro da variável.

3.8 Operações com Variáveis

Podemos utilizar variáveis em cálculos.

```
1  programa {  
2      funcao inicio() {  
3          //algoritmo "Soma"  
4        
5      inteiro n1, n2, resultado  
6        
7      // Entrada de dados  
8      leia(n1)  
9      leia(n2)  
10     resultado = n1 + n2  
11       
12     // Saída de dados  
13       
14     escreva(resultado)  
15       
16     }  
17 }  
18
```

Se o usuário informar:

10

15

Resultado:

25

3.9 Exemplo: Cálculo da Média

algoritmo "Media"

var

nota1, nota2, media : real

inicio

escreva("Digite a primeira nota: ")

leia(nota1)

escreva("Digite a segunda nota: ")

leia(nota2)

```
media <- (nota1 + nota2) / 2
```

```
escreval("Media: ", media)
```

```
fimalgoritmo
```

Exemplo de execução:

Digite a primeira nota: 8

Digite a segunda nota: 6

Media: 7

3.10 Constantes

Alguns valores nunca mudam durante a execução.

Nesses casos utilizamos constantes.

Exemplo:

```
const
```

```
PI = 3.1415
```

A constante PI não deve ser alterada.

Constantes tornam o código mais organizado e seguro.

3.11 Teste de Mesa

O teste de mesa consiste em simular a execução do algoritmo manualmente.

Exemplo:

```
algoritmo "Teste"
```

```
var
```

```
x, y : inteiro
```

```
//inicio
```

```
x <- 10
```

```
y <- x + 5
```

```
escreval(y)
```

```
fimalgoritmo
```

Simulação:

Passo	x	y
Inicial	-	-
$x \leftarrow 10$	10	-
$y \leftarrow x + 5$	10	15
Saída	10	15

Resultado exibido:

15

O teste de mesa é uma das ferramentas mais importantes para identificar erros lógicos.

3.12 Exemplo Completo

Programa para calcular a área de um retângulo.

Exemplo:

Fórmula:

Área = Base × Altura

```
algoritmo "AreaRetangulo"
```

```
var
```

```
base, altura, area : real
```

```
//inicio
```

```
escreva("Digite a base: ")
```

```
leia(base)
```

```
escreva("Digite a altura: ")
```

```
leia(altura)
```

```
area <- base * altura
```

```
escreval("Area = ", area)
```

```
fimalgoritmo
```


Digite a base: 5

Digite a altura: 4

Area = 20

Boas Práticas

Ao trabalhar com variáveis:

- ✓ Utilize nomes significativos.
- ✓ Evite abreviações excessivas.
- ✓ Declare cada variável com o tipo correto.
- ✓ Inicialize variáveis sempre que possível.
- ✓ Evite reutilizar variáveis para finalidades diferentes.

Exemplo recomendado:

quantidade_alunos

media_turma

salario_funcionario

Exemplo não recomendado:

a

b

x

y

Resumo do Capítulo

Neste capítulo aprendemos:

- O conceito de variável.
- Como a memória armazena dados.
- Declaração de variáveis.
- Tipos primitivos.
- Atribuição.
- Entrada de dados.
- Operações matemáticas.

- Constantes.
- Teste de mesa.

Esses conceitos serão a base para os próximos capítulos, onde passaremos a realizar cálculos e tomar decisões utilizando os dados armazenados.

Exercícios Propostos

1. Crie um algoritmo que leia um nome e exiba o valor digitado.
2. Crie um algoritmo que leia a idade de uma pessoa e exiba o resultado.
3. Leia dois números inteiros e exiba a soma.
4. Leia dois números reais e exiba a multiplicação.
5. Calcule a média de três notas.
6. Calcule a área de um quadrado.
7. Calcule o perímetro de um retângulo.
8. Leia o salário de um funcionário e exiba o valor informado.
9. Crie um algoritmo que leia a temperatura em Celsius e exiba o valor armazenado.
10. Faça o teste de mesa do seguinte algoritmo:

`x <- 5`

`y <- 10`

`z <- x + y`

11. Crie uma constante chamada TAXA_JUROS com valor 0.05.
12. Desenvolva um programa que calcule o dobro e o triplo de um número informado pelo usuário.

CAPÍTULO 4 – OPERADORES

4.1 Introdução

Até o momento aprendemos a armazenar informações em variáveis. Entretanto, apenas guardar dados não é suficiente para resolver problemas computacionais.

Precisamos manipular esses valores, realizar cálculos e comparações para produzir resultados úteis.

Os operadores são símbolos especiais utilizados para executar operações sobre dados armazenados nas variáveis.

No Portugol, os operadores são divididos em três grupos principais:

- Operadores Aritméticos
- Operadores Relacionais

- Operadores Lógicos

O domínio desses operadores é essencial para o desenvolvimento de algoritmos.

4.2 Operadores Aritméticos

Os operadores aritméticos são utilizados para realizar cálculos matemáticos.

Operador	Função
+	Adição
-	Subtração
*	Multiplicação
/	Divisão
%	Resto da divisão
^	Potenciação

Adição

```
resultado <- 10 + 5
```

Resultado:

15

Subtração

```
resultado <- 10 - 3
```

Resultado:

7

Multiplicação

```
resultado <- 4 * 6
```

Resultado:

24

Divisão

```
resultado <- 20 / 4
```

Resultado:

5

Resto da Divisão

Também chamado de operador módulo.

```
resultado <- 10 % 3
```

Resultado:

1

Explicação:

$$10 \div 3 = 3$$

$$\text{Resto} = 1$$

Esse operador é muito utilizado para verificar números pares e ímpares.

Exemplo:

```
numero % 2
```

Se o resultado for zero, o número é par.

Potenciação

```
resultado <- 2 ^ 3
```

Resultado:

8

Pois:

$$2 \times 2 \times 2 = 8$$

4.3 Expressões Matemáticas

Uma expressão matemática pode combinar vários operadores.

Exemplo:

```
resultado <- 10 + 5 * 2
```

Resultado:

20

E não:

30

Isso ocorre devido à precedência dos operadores.

4.4 Precedência dos Operadores

Quando há vários operadores na mesma expressão, o computador segue uma ordem de execução.

Ordem de Prioridade

1. Parênteses
2. Potência
3. Multiplicação e Divisão
4. Adição e Subtração

Exemplo:

```
resultado <- 10 + 5 * 2
```

Primeiro:

$$5 \times 2 = 10$$

Depois:

$$10 + 10 = 20$$

Exemplo com parênteses:

```
resultado <- (10 + 5) * 2
```

Primeiro:

$$10 + 5 = 15$$

Depois:

$$15 \times 2 = 30$$

Resultado final:

30

4.5 Operadores Relacionais

Os operadores relacionais são utilizados para comparar valores.

O resultado sempre será:

VERDADEIRO

ou

FALSO

Operador	Significado
=	Igual

Operador	Significado
<>	Diferente
>	Maior
<	Menor
>=	Maior ou igual
<=	Menor ou igual

Igual

10 = 10

Resultado:

VERDADEIRO

Diferente

10 <> 5

Resultado:

VERDADEIRO

Maior

20 > 10

Resultado:

VERDADEIRO

Menor

5 < 8

Resultado:

VERDADEIRO

Maior ou Igual

10 >= 10

Resultado:

VERDADEIRO

Menor ou Igual

5 <= 8

Resultado:

VERDADEIRO

4.6 Expressões Relacionais

As comparações normalmente envolvem variáveis.

Exemplo:

idade <- 20

idade >= 18

Resultado:

VERDADEIRO

Outro exemplo:

nota <- 5

nota >= 7

Resultado:

FALSO

4.7 Operadores Lógicos

Os operadores lógicos permitem combinar condições.

São fundamentais para a tomada de decisões.

Operador	Função
E	Conjunção
OU	Disjunção
NAO	Negação

4.8 Operador E

O operador E retorna VERDADEIRO apenas quando todas as condições forem verdadeiras.

Tabela Verdade:

A B A E B

F F F

F V F

V F F

V V V

Exemplo:

idade >= 18 E possui_cnh = VERDADEIRO

A pessoa deve atender ambas as condições.

4.9 Operador OU

Retorna VERDADEIRO quando pelo menos uma condição é verdadeira.

Tabela Verdade:

A B A OU B

F F F

F V V

V F V

V V V

Exemplo:

nota >= 7 OU possui_recuperacao = VERDADEIRO

Basta uma das condições ser verdadeira.

4.10 Operador NAO

Inverte o valor lógico.

Exemplo:

NAO VERDADEIRO

Resultado:

FALSO

Outro exemplo:

NAO FALSO

Resultado:

VERDADEIRO

4.11 Expressões Lógicas Completas

Exemplo:

idade <- 20

possui_cnh <- VERDADEIRO

Expressão:

(idade >= 18) E (possui_cnh = VERDADEIRO)

Resultado:

VERDADEIRO

Outro exemplo:

idade <- 16

possui_autorizacao <- VERDADEIRO

Expressão:

(idade >= 18) OU (possui_autorizacao = VERDADEIRO)

Resultado:

VERDADEIRO

4.12 Aplicação Prática

Programa para calcular a média de um aluno.

algoritmo "MediaAluno"

var nota1, nota2, media : real

inicio

leia(nota1)

leia(nota2)

media = (nota1 + nota2) / 2

escreval("Media: ", media)

fimalgoritmo

Nesse algoritmo utilizamos:

- Soma (+)
 - Divisão (/)
 - Atribuição (<-)
-

4.13 Teste de Mesa

Algoritmo:

$x \leftarrow 8$

$y \leftarrow 4$

resultado $\leftarrow x * y + 2$

Simulação:

Passo	x	y	resultado
-------	---	---	-----------

Inicial	-	-	-
---------	---	---	---

$x \leftarrow 8$	8	-	-
------------------	---	---	---

$y \leftarrow 4$	8	4	-
------------------	---	---	---

$x * y$	8	4	32
---------	---	---	----

$+ 2$	8	4	34
-------	---	---	----

Resultado final:

34

Boas Práticas

- ✓ Utilize parênteses quando a expressão for complexa.
- ✓ Evite expressões excessivamente grandes.
- ✓ Divida cálculos complexos em etapas.
- ✓ Utilize nomes significativos para variáveis.
- ✓ Teste expressões utilizando teste de mesa.

Resumo do Capítulo

Neste capítulo aprendemos:

- Operadores aritméticos.
- Operadores relacionais.
- Operadores lógicos.
- Precedência das operações.
- Tabelas verdade.
- Construção de expressões matemáticas e lógicas.

Esses conceitos serão utilizados no próximo capítulo para criar algoritmos capazes de tomar decisões através das estruturas condicionais.

Exercícios Propostos

1. Calcule a soma de dois números informados pelo usuário.
2. Calcule a diferença entre dois números.
3. Calcule o produto de dois números.
4. Calcule a média de três notas.
5. Determine o resto da divisão entre dois números.
6. Verifique se um número é par utilizando o operador módulo.
7. Avalie a expressão:

$10 + 5 * 2$

8. Avalie a expressão:

$(10 + 5) * 2$

9. Determine o resultado lógico:

$20 > 10$

10. Determine o resultado lógico:

$5 >= 8$

11. Construa uma tabela verdade para o operador E.
12. Construa uma tabela verdade para o operador OU.
13. Explique a diferença entre E e OU.
14. Explique a função do operador NAO.
15. Faça o teste de mesa:

$x < -5$

$y < -3$

$z < -x + y * 2$

CAPÍTULO 5 – ESTRUTURAS CONDICIONAIS

5.1 Introdução

Nos capítulos anteriores aprendemos a armazenar dados em variáveis e realizar operações matemáticas e lógicas.

Entretanto, muitos problemas exigem que o programa tome decisões com base em determinadas condições.

Por exemplo:

- Verificar se um aluno foi aprovado.

- Determinar se uma pessoa é maior de idade.
- Conceder descontos em uma compra.
- Autorizar ou negar acesso a um sistema.

Para resolver situações como essas utilizamos as estruturas condicionais.

As estruturas condicionais permitem alterar o fluxo de execução de um algoritmo de acordo com o resultado de uma expressão lógica.

5.2 O Conceito de Tomada de Decisão

Imagine a seguinte situação:

Uma escola possui a regra:

Média maior ou igual a 7 → Aprovado

Média menor que 7 → Reprovado

O computador deverá analisar a média do aluno e decidir qual mensagem exibir.

Esse processo é chamado de tomada de decisão.

Em Portugal, essa decisão é realizada através da estrutura:

SE ... ENTAO ... SENAO

5.3 Estrutura Condicional Simples

A estrutura condicional simples executa um bloco de comandos apenas quando uma condição é verdadeira.

Sintaxe:

se (condicao) entao

comandos

fimse

Exemplo:

algoritmo "MaiorIdade"

var

idade : inteiro

inicio

```
leia(idade)
```

```
se (idade >= 18) entao  
    escreval("Maior de idade")  
fimse
```

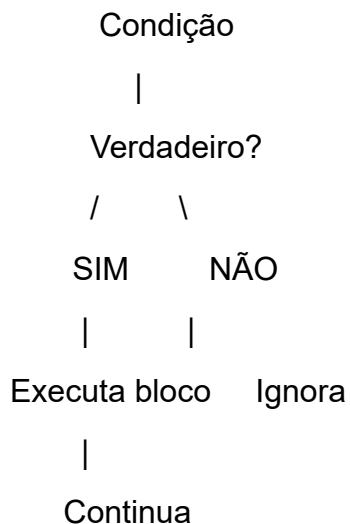
fimalgoritmo

Se a idade for 18 ou maior, a mensagem será exibida.

Caso contrário, nada acontecerá.

5.4 Fluxo da Condicional Simples

Representação:



5.5 Estrutura Condicional Composta

Em muitos casos precisamos executar ações diferentes dependendo do resultado da condição.

Para isso utilizamos:

SE ... SENAO

Sintaxe:

```
se (condicao) entao
```

```
    comandos
```

senao

comandos

fimse

Exemplo:

algoritmo "Aprovacao"

var

media : real

inicio

leia(media)

se (media $\geq$ 7) entao

 escreval("Aprovado")

senao

 escreval("Reprovado")

fimse

fimalgoritmo

5.6 Exemplo Prático – Sistema de Desconto

algoritmo "Desconto"

var

valor_compra : real

inicio

leia(valor_compra)

```
se (valor_compra >= 100) entao
    escreval("Cliente recebeu desconto")
senao
    escreval("Sem desconto")
fimse

finalgoritmo
```

5.7 Condições Compostas

Frequentemente precisamos combinar várias condições.

Para isso utilizamos os operadores lógicos.

Exemplo:

```
algoritmo "Habilitacao"
```

```
var
```

```
idade : inteiro
```

```
possui_documento : logico
```

```
inicio
```

```
leia(idade)
```

```
leia(possui_documento)
```

```
se (idade >= 18) e (possui_documento = VERDADEIRO) entao
```

```
    escreval("Cadastro autorizado")
```

```
fimse
```

```
finalgoritmo
```

Nesse caso as duas condições precisam ser verdadeiras.

5.8 Operador OU em Condicionais

Exemplo:

algoritmo "Beneficio"

var

idoso, deficiente : logico

inicio

leia(idoso)

leia(deficiente)

se (idoso = VERDADEIRO) ou (deficiente = VERDADEIRO) entao

 escreval("Benefício concedido")

fimse

fimalgoritmo

Apenas uma das condições precisa ser verdadeira.

5.9 Condicionais Encadeadas

Em algumas situações existem mais de duas possibilidades.

Exemplo:

Média $\geq 7 \rightarrow$ Aprovado

Média $\geq 5 \rightarrow$ Recuperação

Média $< 5 \rightarrow$ Reprovado

Para resolver esse problema utilizamos condicionais encadeadas.

Sintaxe:

se (condicao1) entao

senao

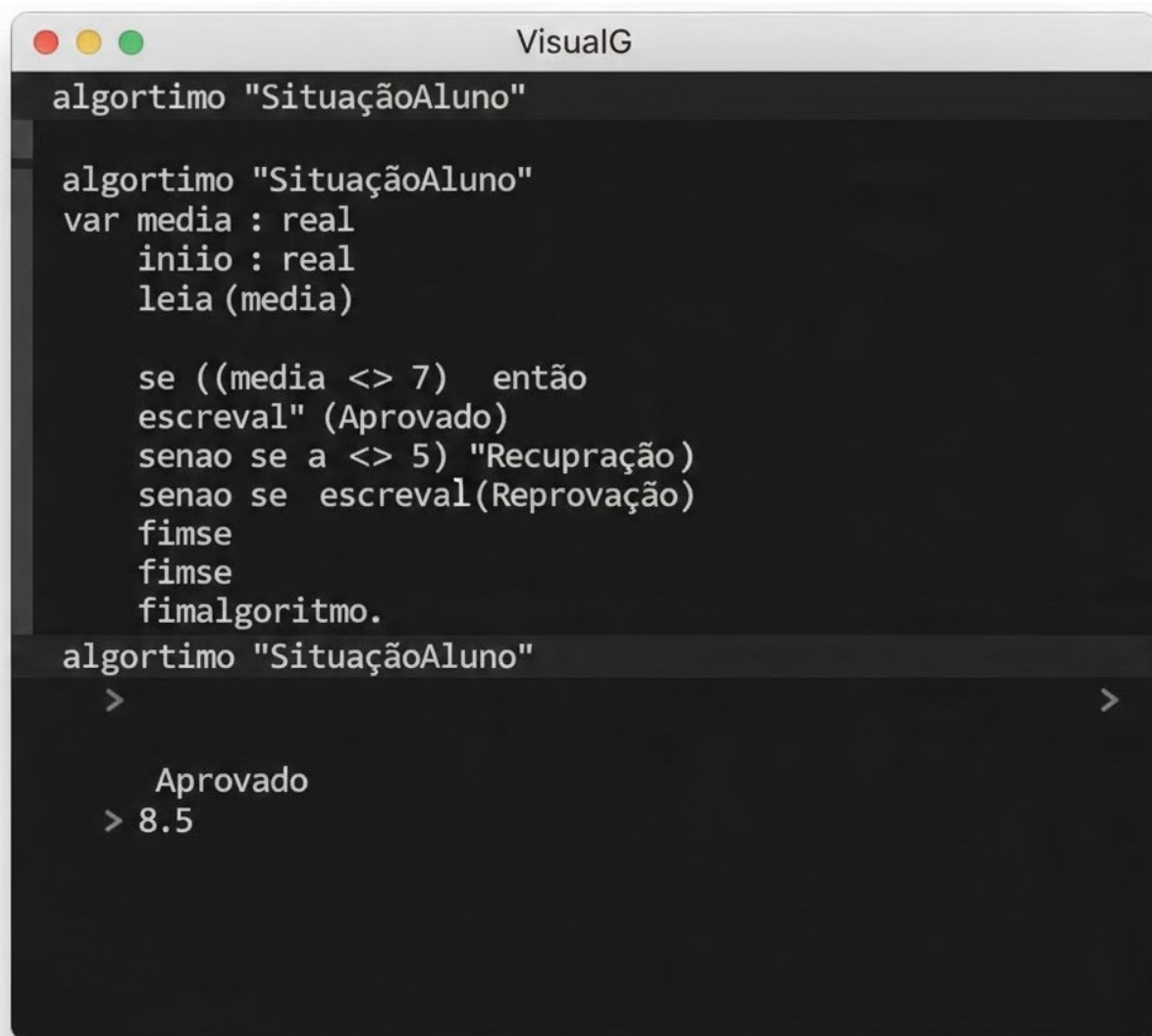
 se (condicao2) entao

senao

fimse

fimse

5.10 Exemplo – Situação do Aluno



```
algoritmo "SituaçãoAluno"

algoritmo "SituaçãoAluno"
var media : real
    iniio : real
    leia (media)

    se ((media <> 7) então
        escreval" (Aprovado)
    senao se a <> 5) "Recupração)
    senao se escreval(Reprovação)
    fimse
    fimse
    fimalgoritmo.

algoritmo "SituaçãoAluno"
>
>
    Aprovado
> 8.5
```

5.11 Estrutura Escolha Caso

Quando existem muitas alternativas, utilizar vários comandos SE pode tornar o código difícil de ler.

Nesses casos utilizamos:

ESCOLHA CASO

Essa estrutura funciona de maneira semelhante a um menu.

Sintaxe:

```
escolha variavel
```

```
caso valor1
```

```
    comandos
```

```
caso valor2
```

```
    comandos
```

```
outrocaso
```

```
    comandos
```

```
fimescolha
```

5.12 Exemplo – Menu de Opções

```
algoritmo "Menu"
```

```
var
```

```
opcao : inteiro
```

```
inicio
```

```
    escreval("1 - Cadastrar")
```

```
    escreval("2 - Consultar")
```

```
    escreval("3 - Excluir")
```

```
    leia(opcao)
```

```
    escolha opcao
```

```
    caso 1
```

```
escreval("Cadastro selecionado")
```

caso 2

```
escreval("Consulta selecionada")
```

caso 3

```
escreval("Exclusao selecionada")
```

outro caso

```
escreval("Opcao invalida")
```

fimescolha

fimalgoritmo

5.13 Teste de Mesa

Algoritmo:

media <- 8

se (media >= 7) entao

```
    escreval("Aprovado")
```

senao

```
    escreval("Reprovado")
```

fimse

Simulação:

Passo	Média	Condição
Inicial	8	-
media >= 7	8	VERDADEIRO
Executa bloco	8	Aprovado

Resultado:

Aprovado

5.14 Outro Teste de Mesa

idade <= 15

se (idade >= 18) entao

 escreval("Maior")

senao

 escreval("Menor")

fimse

Simulação:

Passo	Idade	Resultado
Inicial	15	-
idade >= 18	15	FALSO
Executa senao	15	Menor

Resultado:

Menor

5.15 Aplicações Reais

As estruturas condicionais estão presentes em praticamente todos os sistemas computacionais.

Exemplos:

Sistema Bancário:

Saldo suficiente?

SIM → Autoriza saque

NÃO → Recusa saque

Sistema Escolar:

Média >= 7?

SIM → Aprova

NÃO → Reprova

Sistema de Login:

Senha correta?

SIM → Permite acesso

NÃO → Bloqueia acesso

Boas Práticas

- ✓ Utilize condições simples e objetivas.
- ✓ Evite excesso de aninhamentos.
- ✓ Utilize ESCOLHA quando houver muitas alternativas.
- ✓ Faça testes de mesa antes de executar o programa.
- ✓ Organize o código com indentação adequada.

Exemplo:

```
se (idade >= 18) entao
```

```
    escreval("Permitido")
```

```
senao
```

```
    escreval("Negado")
```

```
fimse
```

Resumo do Capítulo

Neste capítulo aprendemos:

- Estruturas condicionais simples.
- Estruturas condicionais compostas.
- Condições compostas com E e OU.
- Condicionais encadeadas.
- Estrutura ESCOLHA CASO.
- Teste de mesa.
- Aplicações práticas das decisões em algoritmos.

Esses conceitos permitem que os programas tomem decisões inteligentes com base nas informações fornecidas pelo usuário.

Exercícios Propostos

1. Leia a idade de uma pessoa e informe se ela é maior de idade.

2. Leia uma nota e informe se o aluno foi aprovado.
3. Leia um número e informe se ele é positivo.
4. Leia um número e informe se ele é par ou ímpar.
5. Leia dois números e informe qual é o maior.
6. Leia três números e informe qual é o maior.
7. Crie um programa que calcule a média de um aluno e informe:
 - Aprovado
 - Recuperação
 - Reprovado
8. Leia o salário de um funcionário e informe se ele receberá bônus.
9. Crie um menu utilizando ESCOLHA CASO com quatro opções.
10. Faça o teste de mesa do algoritmo:

```
idade <- 20
se (idade >= 18) entao
  escreval("Maior")
senao
  escreval("Menor")
fimse
```
11. Desenvolva um algoritmo que verifique se uma pessoa pode votar.
12. Desenvolva um algoritmo que determine se um candidato foi aprovado em um concurso com base na nota mínima exigida.

CAPÍTULO 6 – ESTRUTURAS DE REPETIÇÃO

6.1 Introdução

Até agora, todos os algoritmos desenvolvidos executavam cada instrução apenas uma vez.

Entretanto, existem situações em que precisamos repetir uma mesma ação diversas vezes.

Imagine os seguintes exemplos:

- Exibir os números de 1 até 100.
- Ler as notas de uma turma com 30 alunos.
- Calcular a soma de vários valores.
- Processar milhares de registros em um sistema.

Executar essas tarefas manualmente seria inviável.

Para resolver esse problema utilizamos as estruturas de repetição, também conhecidas como laços ou loops.

Essas estruturas permitem executar um conjunto de instruções várias vezes de forma automática.

6.2 O Conceito de Repetição

Considere a seguinte situação:

Desejamos exibir os números de 1 a 5.

Sem repetição:

escreval(1)

escreval(2)

escreval(3)

escreval(4)

escreval(5)

Embora funcione, essa solução não é eficiente.

Imagine exibir os números de 1 até 1000.

A solução correta é utilizar uma estrutura de repetição.

6.3 Estrutura ENQUANTO

A estrutura ENQUANTO executa um bloco de comandos enquanto uma condição for verdadeira.

Sintaxe:

enquanto (condicao) faca

comandos

fimenquanto

Funcionamento:

Verifica condição

|

V

Verdadeira?

/ \
SIM NÃO
| |
Executa Encerra
Bloco
|
Retorna ao início

6.4 Primeiro Exemplo com ENQUANTO

algoritmo "Contagem"

var

contador : inteiro

início

contador <- 1

enquanto (contador <= 5) faça

 escreval(contador)

 contador <- contador + 1

fimenquanto

fimalgoritmo

Resultado:

1

2

3

4

6.5 O Conceito de Contador

Um contador é uma variável utilizada para controlar quantas vezes um laço será executado.

Exemplo:

```
contador <- contador + 1
```

Essa instrução incrementa o valor da variável.

Se:

```
contador = 5
```

Após a execução:

```
contador = 6
```

Os contadores são fundamentais em estruturas de repetição.

6.6 Teste de Mesa do ENQUANTO

Algoritmo:

```
contador <- 1
```

```
enquanto (contador <= 3) faça
```

```
    escreval(contador)
```

```
    contador <- contador + 1
```

```
fimenquanto
```

Simulação:

Iteraçã o	Contador
Inicial	1
1 ^a	1
2 ^a	2
3 ^a	3
Final	4

Saída:

1

2

3

6.7 Acumuladores

Um acumulador é uma variável utilizada para armazenar resultados parciais.

Exemplo:

Desejamos somar os números de 1 até 5.

algoritmo "Soma"

var

numero, soma : inteiro

inicio

numero <- 1

soma <- 0

enquanto (numero <= 5) faca

 soma <- soma + numero

 numero <- numero + 1

fimenquanto

escreval("Resultado: ", soma)

fimalgoritmo

Resultado:

15

6.8 Estrutura REPITA ATÉ

A estrutura REPITA executa o bloco primeiro e verifica a condição apenas ao final.

Sintaxe:

```
repita
```

```
    comandos
```

```
ate (condicao)
```

Diferença principal:

Estrutura	Verifica Antes?
ENQUANTO	Sim
REPITA	Não

6.9 Exemplo com REPITA

algoritmo "ExemploRepita"

```
var
```

```
    contador : inteiro
```

```
inicio
```

```
    contador <- 1
```

```
    repita
```

```
        escreval(contador)
```

```
        contador <- contador + 1
```

```
    ate (contador > 5)
```

```
fimalgoritmo
```

Resultado:

1
2
3
4
5

6.10 Quando Utilizar REPITA

REPITA é útil quando desejamos garantir que o bloco seja executado pelo menos uma vez.

Exemplo:

Menu de sistema:

Exibe menu

Recebe opção

Verifica se é válida

Mesmo que a opção seja incorreta, o menu será apresentado ao menos uma vez.

6.11 Estrutura PARA

A estrutura PARA é utilizada quando sabemos exatamente quantas repetições serão necessárias.

Sintaxe:

para contador de inicio ate fim faca

comandos

fimpara

6.12 Primeiro Exemplo com PARA

algoritmo "Contagem"

var

i : inteiro

inicio

para i de 1 ate 10 faca

 escreval(i)

fimpara

fimalgoritmo

Resultado:

1

2

3

4

5

6

7

8

9

10

6.13 Comparação Entre as Estruturas

ENQUANTO

Utilizado quando não sabemos exatamente quantas repetições ocorrerão.

Exemplo:

Enquanto houver clientes na fila.

REPITA

Utilizado quando o bloco deve executar pelo menos uma vez.

Exemplo:

Menu principal.

PARA

Utilizado quando a quantidade de repetições é conhecida.

Exemplo:

Exibir números de 1 até 100.

6.14 Exemplo Prático – Tabuada

algoritmo "Tabuada"

var

numero, i : inteiro

inicio

leia(numero)

para i de 1 ate 10 faca

 escreval(numero, " x ", i, " = ", numero * i)

fimpara

fimalgoritmo

6.15 Exemplo Prático – Soma de Notas

algoritmo "MediaTurma"

var

i : inteiro

nota, soma : real

inicio

```
soma <- 0
```

```
para i de 1 ate 5 faca
```

```
    leia(nota)
```

```
    soma <- soma + nota
```

```
fimpara
```

```
escreval("Total: ", soma)
```

```
fimalgoritmo
```

6.16 Laço Infinito

Um laço infinito ocorre quando a condição de parada nunca é alcançada.

Exemplo:

```
contador <- 1
```

```
enquanto (contador <= 10) faca
```

```
    escreval(contador)
```

```
fimenquanto
```

Problema:

O contador nunca é incrementado.

Consequência:

```
1
```

```
1
```

```
1
```

```
1
```

1

...

O programa ficará preso indefinidamente.

6.17 Boas Práticas

- ✓ Sempre verifique a condição de parada.
 - ✓ Inicialize corretamente os contadores.
 - ✓ Utilize PARA quando souber a quantidade de repetições.
 - ✓ Utilize nomes significativos para variáveis.
 - ✓ Faça testes de mesa em algoritmos complexos.
 - ✓ Evite laços infinitos.
-

6.18 Aplicações Reais

As estruturas de repetição estão presentes em praticamente todos os sistemas computacionais.

Exemplos:

Sistema Bancário:

Processar milhares de transações.

Sistema Escolar:

Calcular médias de todos os alunos.

Sistema de Estoque:

Percorrer todos os produtos cadastrados.

Jogos:

Atualizar a tela dezenas de vezes por segundo.

Resumo do Capítulo

Neste capítulo aprendemos:

- Estrutura ENQUANTO.
- Estrutura REPITA ATÉ.
- Estrutura PARA.
- Contadores.
- Acumuladores.

- Teste de mesa.
- Laços infinitos.
- Aplicações práticas das repetições.

As estruturas de repetição são fundamentais para automatizar tarefas e processar grandes quantidades de dados.

Exercícios Propostos

1. Exiba os números de 1 até 20 utilizando ENQUANTO.
2. Exiba os números de 20 até 1.
3. Calcule a soma dos números de 1 até 100.
4. Exiba apenas os números pares de 1 até 50.
5. Exiba apenas os números ímpares de 1 até 50.
6. Desenvolva uma tabuada utilizando PARA.
7. Leia 10 números e calcule a soma.
8. Leia 10 números e determine o maior valor.
9. Calcule o fatorial de um número.
10. Conte quantos alunos possuem nota maior ou igual a 7.
11. Desenvolva um menu utilizando REPITA ATÉ.
12. Faça o teste de mesa do seguinte algoritmo:
 contador <- 1
 enquanto (contador <= 5) faça
 escreva(contador)
 contador <- contador + 1
 fimenquanto
13. Desenvolva um algoritmo que leia números até que o usuário digite zero.
14. Calcule a média de uma turma com 20 alunos.
15. Desenvolva um programa que conte quantos números pares foram digitados pelo usuário.

CAPÍTULO 7 – VETORES E MATRIZES

7.1 Introdução

Nos capítulos anteriores utilizamos variáveis para armazenar informações individuais.

Por exemplo:

```
idade <- 18
```

ou

```
nome <- "Carlos"
```

Entretanto, imagine uma situação em que precisamos armazenar as notas de 30 alunos.

Uma solução seria:

```
nota1 <- 8
```

```
nota2 <- 7
```

```
nota3 <- 10
```

...

```
nota30 <- 9
```

Além de trabalhosa, essa abordagem é pouco prática e difícil de manter.

Para resolver esse problema utilizamos estruturas capazes de armazenar vários valores sob um único identificador.

Essas estruturas são chamadas de Vetores e Matrizes.

7.2 O que é um Vetor?

Um vetor é uma estrutura de dados capaz de armazenar vários valores do mesmo tipo.

Cada valor ocupa uma posição chamada índice.

Representação:

Índice: 1 2 3 4 5

Notas: 8 7 10 6 9

Nesse exemplo:

```
notas[1] = 8
```

```
notas[2] = 7
```

```
notas[3] = 10
```

```
notas[4] = 6
```

```
notas[5] = 9
```

7.3 Declaração de Vetores

Sintaxe:

`vetor[inicio..fim]` de tipo

Exemplo:

```
var
```

```
notas : vetor[1..5] de real
```

Nesse caso foram reservadas cinco posições para armazenar números reais.

7.4 Acessando Elementos

Podemos acessar qualquer posição utilizando o índice.

Exemplo:

```
notas[1] <- 8.5
```

```
notas[2] <- 7.0
```

```
notas[3] <- 9.5
```

Para exibir:

```
escreval(notas[1])
```

Resultado:

```
8.5
```

7.5 Preenchendo um Vetor Manualmente

algoritmo "Notas"

```
var
```

```
notas : vetor[1..5] de real
```

```
inicio
```

```
notas[1] <- 8
```

```
notas[2] <- 7
```

```
notas[3] <- 9
```

```
notas[4] <- 6
```

```
notas[5] <- 10
```

```
escreval(notas[3])
```

```
fimalgoritmo
```

Resultado:

9

7.6 Utilizando Laços com Vetores

A principal vantagem dos vetores aparece quando utilizamos estruturas de repetição.

Exemplo:

algoritmo "CadastroNotas"

```
var
```

```
notas : vetor[1..5] de real
```

```
i : inteiro
```

```
inicio
```

```
para i de 1 ate 5 faca
```

```
    escreva("Digite a nota: ")
```

```
    leia(notas[i])
```

```
fimpara
```

```
fimalgoritmo
```

Nesse caso o mesmo código atende todas as posições do vetor.

7.7 Exibindo um Vetor

algoritmo "ExibirVetor"

var

numeros : vetor[1..5] de inteiro

i : inteiro

inicio

para i de 1 ate 5 faca

 leia(numeros[i])

fimpara

para i de 1 ate 5 faca

 escreval(numeros[i])

fimpara

fimalgoritmo

7.8 Calculando a Média de uma Turma

algoritmo "MediaTurma"

var

notas : vetor[1..5] de real

i : inteiro

soma, media : real

inicio

soma <- 0

para i de 1 ate 5 faca

 leia(notas[i])

 soma <- soma + notas[i]

fimpara

media <- soma / 5

escreval("Media: ", media)

fimalgoritmo

7.9 Encontrando o Maior Valor

Uma tarefa muito comum consiste em localizar o maior elemento de um vetor.

Exemplo:

algoritmo "MaiorValor"

var

numeros : vetor[1..5] de inteiro

i, maior : inteiro

inicio

para i de 1 ate 5 faca

leia(numeros[i])

fimpara

maior <- numeros[1]

para i de 2 ate 5 faca

se (numeros[i] > maior) entao

maior <- numeros[i]

fimse

fimpara

escreval("Maior valor: ", maior)

fimalgoritmo

7.10 Busca Sequencial

Buscar informações é uma das operações mais realizadas em sistemas computacionais.

Exemplo:

Desejamos verificar se determinado número existe em um vetor.

algoritmo "Busca"

var

numeros : vetor[1..5] de inteiro

i, procurado : inteiro

encontrado : logico

inicio

encontrado <- FALSO

leia(procurado)

para i de 1 ate 5 faca

se (numeros[i] = procurado) entao

encontrado <- VERDADEIRO

fimse

fimpara

escreval(encontrado)

fimalgoritmo

7.11 O que é uma Matriz?

Uma matriz é uma estrutura bidimensional.

Enquanto o vetor possui apenas uma dimensão, a matriz possui linhas e colunas.

Representação:

Colunas

1 2 3

Linha 1 5 8 4

Linha 2 2 9 7

Linha 3 1 6 3

7.12 Declaração de Matrizes

Sintaxe:

vetor[linhas, colunas] de tipo

Exemplo:

var

tabuleiro : vetor[1..3,1..3] de inteiro

7.13 Acessando Elementos da Matriz

tabuleiro[1,1] <- 5

tabuleiro[2,3] <- 7

tabuleiro[3,2] <- 6

Representação:

5 0 0

0 0 7

0 6 0

7.14 Preenchendo uma Matriz

Para percorrer uma matriz utilizamos dois laços.

Um para as linhas e outro para as colunas.

algoritmo "Matriz"

var

matriz : vetor[1..3,1..3] de inteiro

linha, coluna : inteiro

inicio

para linha de 1 ate 3 faca

para coluna de 1 ate 3 faca

leia(matriz[linha,coluna])

fimpara

fimpara

fimalgoritmo

7.15 Exibindo uma Matriz

algoritmo "MostrarMatriz"

var

matriz : vetor[1..3,1..3] de inteiro

linha, coluna : inteiro

inicio

para linha de 1 ate 3 faca

para coluna de 1 ate 3 faca

escreva(matriz[linha,coluna], " ")

fimpara

escreval("")

fimpara

finalgoritmo

7.16 Aplicações Práticas

Vetores e matrizes estão presentes em praticamente todos os sistemas modernos.

Exemplos:

Sistema Escolar:

Lista de alunos

Lista de notas

Sistema Bancário:

Lista de clientes

Lista de contas

Jogos:

Mapas

Tabuleiros

Inventários

Planilhas:

Linhas e colunas

Bancos de Dados:

Tabelas

7.17 Teste de Mesa

Considere:

```
vetor[1] <- 10
```

```
vetor[2] <- 20
```

```
vetor[3] <- 30
```

Memória:

Índice	Valor
1	10
2	20
3	30

Comando:

```
escreval(vetor[2])
```

Resultado:

20

7.18 Boas Práticas

- ✓ Utilize laços para percorrer vetores.
- ✓ Evite repetir código.
- ✓ Sempre valide os índices.
- ✓ Utilize nomes significativos.
- ✓ Faça testes de mesa antes da execução.
- ✓ Inicialize variáveis de controle.

Resumo do Capítulo

Neste capítulo aprendemos:

- O conceito de vetor.
- Declaração de vetores.
- Acesso a elementos.
- Percorrimento com laços.
- Busca sequencial.
- O conceito de matriz.
- Linhas e colunas.
- Manipulação de matrizes.

- Aplicações práticas.

Essas estruturas permitem armazenar e manipular grandes quantidades de dados de forma organizada.

Exercícios Propostos

1. Crie um vetor para armazenar 10 números inteiros.
2. Leia e exiba os valores de um vetor com 5 posições.
3. Calcule a soma dos elementos de um vetor.
4. Calcule a média dos elementos de um vetor.
5. Determine o maior valor armazenado em um vetor.
6. Determine o menor valor armazenado em um vetor.
7. Conte quantos números pares existem em um vetor.
8. Conte quantos números ímpares existem em um vetor.
9. Desenvolva uma busca sequencial.
10. Crie uma matriz 3x3.
11. Leia os elementos de uma matriz 3x3.
12. Exiba todos os elementos da matriz.
13. Calcule a soma dos elementos de uma matriz.
14. Conte quantos valores positivos existem em uma matriz.
15. Desenvolva um programa que simule um boletim escolar utilizando vetores.

CAPÍTULO 8 – PROCEDIMENTOS E FUNÇÕES

8.1 Introdução

Até este momento, todos os algoritmos desenvolvidos foram escritos dentro de um único bloco principal.

Embora essa abordagem funcione para programas pequenos, ela se torna inadequada conforme o sistema cresce.

Imagine um programa com centenas ou milhares de linhas.

Nessas situações surgem problemas como:

- Dificuldade de manutenção.
- Repetição de código.
- Aumento da probabilidade de erros.

- Baixa legibilidade.

Para resolver essas dificuldades utilizamos a modularização.

A modularização consiste em dividir um programa em partes menores e independentes chamadas módulos.

Os principais módulos em Portugol são:

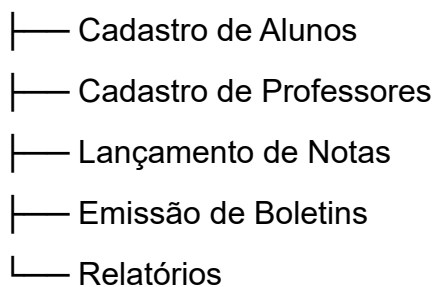
- Procedimentos
- Funções

8.2 O Conceito de Modularização

Considere um sistema escolar.

Ele pode ser dividido em diversos módulos:

Sistema Escolar



Cada módulo possui uma responsabilidade específica.

Essa divisão torna o sistema mais organizado e fácil de manter.

8.3 O que é um Procedimento?

Um procedimento é um conjunto de comandos agrupados para realizar uma tarefa específica.

Sua principal característica é que ele não retorna valores.

Sintaxe:

```
procedimento NomeProcedimento()
```

inicio

comandos

fimprocedimento

8.4 Primeiro Procedimento

Exemplo:

procedimento ExibirMensagem()

inicio

 escreval("Bem-vindo ao sistema")

fimprocedimento

Esse procedimento apenas exibe uma mensagem.

8.5 Chamando um Procedimento

Após criar o procedimento, podemos utilizá-lo dentro do algoritmo principal.

algoritmo "Teste"

procedimento ExibirMensagem()

inicio

 escreval("Bem-vindo ao sistema")

fimprocedimento

inicio

 ExibirMensagem()

fimalgoritmo

Resultado:

Bem-vindo ao sistema

8.6 Vantagens dos Procedimentos

Imagine que uma mensagem precise aparecer em vários locais do sistema.

Sem procedimento:

```
escreval("Sistema Escolar")
```

```
escreval("Sistema Escolar")
```

```
escreval("Sistema Escolar")
```

Com procedimento:

```
ExibirCabecalho()
```

```
ExibirCabecalho()
```

```
ExibirCabecalho()
```

O código fica mais organizado e fácil de modificar.

8.7 Procedimentos com Parâmetros

Muitas vezes precisamos enviar informações para o procedimento.

Para isso utilizamos parâmetros.

Exemplo:

```
procedimento Saudacao(nome : caractere)
```

```
inicio
```

```
    escreval("Olá ", nome)
```

```
fimprocedimento
```

Chamada:

```
Saudacao("Maria")
```

Resultado:

```
Olá Maria
```

8.8 Múltiplos Parâmetros

Um procedimento pode receber vários valores.

```
procedimento ExibirAluno(nome : caractere; idade : inteiro)
```

```
inicio
```

```
    escreval("Nome: ", nome)
```



```
escreval("Idade: ", idade)
```

```
fimprocedimento
```

Chamada:

```
ExibirAluno("Carlos", 20)
```

8.9 O que é uma Função?

Uma função é semelhante a um procedimento.

A diferença é que ela retorna um valor.

Sintaxe:

```
funcao NomeFuncao() : tipo
```

inicio

 retorne valor

fimfuncao

8.10 Primeira Função

```
funcao Dobro(numero : inteiro) : inteiro
```

inicio

 retorne numero * 2

fimfuncao

Utilização:

```
resultado <- Dobro(5)
```

Resultado:

10

8.11 Funções em Expressões

Uma função pode participar de cálculos.

Exemplo:

```
x <- Dobro(10) + 5
```

Primeiro:

```
Dobro(10) = 20
```

Depois:

```
20 + 5 = 25
```

Resultado:

```
25
```

8.12 Função para Calcular Média

```
funcao CalcularMedia(n1, n2 : real) : real
```

```
inicio
```

```
    retorne (n1 + n2) / 2
```

```
fimfuncao
```

Uso:

```
media <- CalcularMedia(8, 6)
```

Resultado:

```
7
```

8.13 Exemplo Completo

```
algoritmo "MediaAluno"
```

```
funcao CalcularMedia(n1, n2 : real) : real
```

```
inicio
```

```
    retorne (n1 + n2) / 2
```

fimfuncao

var

nota1, nota2, media : real

inicio

leia(nota1)

leia(nota2)

media <- CalcularMedia(nota1, nota2)

escreval("Media: ", media)

fimalgoritmo

8.14 Escopo de Variáveis

O escopo determina onde uma variável pode ser utilizada.

Existem dois tipos principais:

Variáveis Globais

São declaradas fora de procedimentos e funções.

Podem ser acessadas por todo o programa.

Exemplo:

var

contador : inteiro

Variáveis Locais

Existem apenas dentro do módulo onde foram declaradas.

Exemplo:

funcao Soma() : inteiro

var

x : inteiro

inicio

x <- 10

retorne x

fimfuncao

A variável x só existe dentro da função.

8.15 Passagem por Valor

Quando enviamos parâmetros para um módulo, normalmente ocorre uma cópia do valor.

Exemplo:

funcao Dobro(numero : inteiro) : inteiro

Se:

numero = 5

A função recebe apenas uma cópia do valor.

A variável original permanece inalterada.

8.16 Teste de Mesa

Função:

funcao Dobro(x : inteiro) : inteiro

inicio

retorne x * 2

fimfuncao

Chamada:

resultado <- Dobro(4)

Simulação:

Etapas	Valor
Entrada	4
$x * 2$	8
Retorno	8
resultado	8

8.17 Aplicação Prática

Calculadora modular.

funcao Somar(a,b : inteiro) : inteiro

inicio

retorne a + b

fimfuncao

funcao Multiplicar(a,b : inteiro) : inteiro

inicio

retorne a * b

fimfuncao

Uso:

resultado1 <- Somar(10,5)

resultado2 <- Multiplicar(10,5)

Resultados:

15

8.18 Benefícios da Modularização

Ao utilizar procedimentos e funções obtemos:

- ✓ Organização.
- ✓ Reutilização de código.
- ✓ Menor quantidade de erros.
- ✓ Facilidade de manutenção.
- ✓ Maior legibilidade.
- ✓ Desenvolvimento mais profissional.

Esses benefícios se tornam fundamentais em sistemas reais.

Boas Práticas

- ✓ Cada função deve possuir apenas uma responsabilidade.
- ✓ Utilize nomes descritivos.
- ✓ Evite funções excessivamente grandes.
- ✓ Utilize parâmetros em vez de variáveis globais sempre que possível.
- ✓ Documente regras complexas.
- ✓ Reutilize módulos já criados.

Exemplo:

CalcularMedia()

CalcularSalario()

ValidarCPF()

Resumo do Capítulo

Neste capítulo aprendemos:

- O conceito de modularização.
- Procedimentos.
- Funções.

- Parâmetros.
- Retorno de valores.
- Escopo de variáveis.
- Passagem de parâmetros.
- Reutilização de código.

A modularização é uma das técnicas mais importantes da programação moderna, permitindo criar sistemas organizados e escaláveis.

Exercícios Propostos

1. Crie um procedimento que exiba seu nome.
2. Crie um procedimento que exiba uma mensagem de boas-vindas.
3. Crie um procedimento que receba um nome como parâmetro.
4. Crie um procedimento que receba nome e idade.
5. Desenvolva uma função que calcule o dobro de um número.
6. Desenvolva uma função que calcule o triplo de um número.
7. Desenvolva uma função que calcule a média de duas notas.
8. Desenvolva uma função que retorne o maior entre dois números.
9. Desenvolva uma função que determine se um número é par.
10. Crie uma calculadora utilizando funções.
11. Faça o teste de mesa da função:

```
funcao Quadrado(x : inteiro) : inteiro  
inicio  
    retorne x * x  
fimfuncao
```
12. Desenvolva um sistema escolar modular utilizando funções para cálculo de média e procedimentos para exibição dos resultados.
13. Crie uma função para calcular a área de um retângulo.
14. Crie uma função para calcular o perímetro de um retângulo.
15. Desenvolva um menu utilizando procedimentos para organizar as opções do sistema.

CAPÍTULO 9 – ALGORITMOS CLÁSSICOS: BUSCA E ORDENAÇÃO

9.1 Introdução

Até este momento aprendemos a armazenar dados em vetores e matrizes, além de utilizar estruturas de repetição e funções.

Entretanto, armazenar informações é apenas parte do problema.

Em sistemas reais, frequentemente precisamos:

- Encontrar informações.
- Organizar dados.
- Localizar registros rapidamente.
- Exibir dados em ordem.

Essas tarefas são realizadas através dos algoritmos de busca e ordenação.

Esses algoritmos estão presentes em praticamente todos os sistemas computacionais modernos.

Exemplos:

- Busca de contatos no celular.
- Pesquisa em lojas virtuais.
- Consulta de clientes em bancos.
- Classificação de notas em sistemas escolares.

9.2 O que é um Algoritmo de Busca?

Um algoritmo de busca tem como objetivo localizar um determinado elemento dentro de uma estrutura de dados.

Exemplo:

Vetor:

Índice: 1 2 3 4 5

Valor: 10 25 30 15 50

Desejamos localizar o valor:

30

O algoritmo de busca percorrerá o vetor até encontrar o elemento desejado.

9.3 Busca Sequencial

A busca sequencial, também chamada de busca linear, é o algoritmo de busca mais simples.

Seu funcionamento consiste em analisar cada posição do vetor até encontrar o valor procurado.

Representação:

10 → 25 → 30 → 15 → 50

↑

Encontrado

9.4 Implementação da Busca Sequencial

```
algoritmo "BuscaSequencial"
var
    numeros : vetor[1..5] de inteiro
    inteiro    inteiro
    i, procurado : inteiro
    encontrado : lógico
inicio
    para i de 1 até 5 faça
        leia(numeros[i])
    fimpara
    escreva("Digite 1 até 5 para o valor procurado: ")
    leia(numerado)
    leia(procurado)
    encontrado ← FALSO
    para i de 1 até 5 faça
        se (numeros[i] = procurado) então
            encontrado ← VERDADEIRO
        fimse
    finsal(encontrado)
fimalgoritmo.
```

> Digite Digite valor procurado:

9.5 Busca com Posição

Em muitos casos não basta saber se o elemento existe.

Também precisamos saber onde ele está.

Exemplo:

```
user@macbook:ss
algoritmo "BuscaPosicao"
algoritmo "  var
    numeros : 11.[5 de inteiro
    var      : vetor[1.5 de inteiro
    i, procurado, : inteiro
    fimpara
    fimpari
inicio para i de 1 até 5 faça
    para i de 1 até 5 faça
        leia(numeros[i])
        leia(procurado)
        se (procurado <= numeros[i]) então
            para i de 5 até 1 faça
                se (numeros[i] = procurado) então
                    posicao <- i
            fimse
        escreva "Posicao: ", posicao
    fimpara
fimalgoritmo.
```

9.6 Teste de Mesa da Busca

Vetor:

10 20 30 40 50

Valor procurado:

30

Simulação:

Iteração	Valor Atual	Encontrado
1	10	Não
2	20	Não
3	30	Sim

Resultado:

Posição 3

9.7 O que é Ordenação?

Ordenar significa organizar elementos seguindo determinado critério.

Exemplos:

Ordem crescente:

2 4 6 8 10

Ordem decrescente:

10 8 6 4 2

A ordenação facilita consultas e análises de dados.

9.8 Por que Ordenar Dados?

Imagine uma escola com centenas de notas.

Lista original:

8 5 10 6 9

Lista ordenada:

5 6 8 9 10

A visualização torna-se muito mais simples.

9.9 Algoritmo Bubble Sort

O Bubble Sort é um dos algoritmos de ordenação mais conhecidos.

Seu funcionamento baseia-se na comparação de elementos vizinhos.

Quando dois elementos estão fora de ordem, eles trocam de posição.

Exemplo:

8 5 10 6

Primeira comparação:

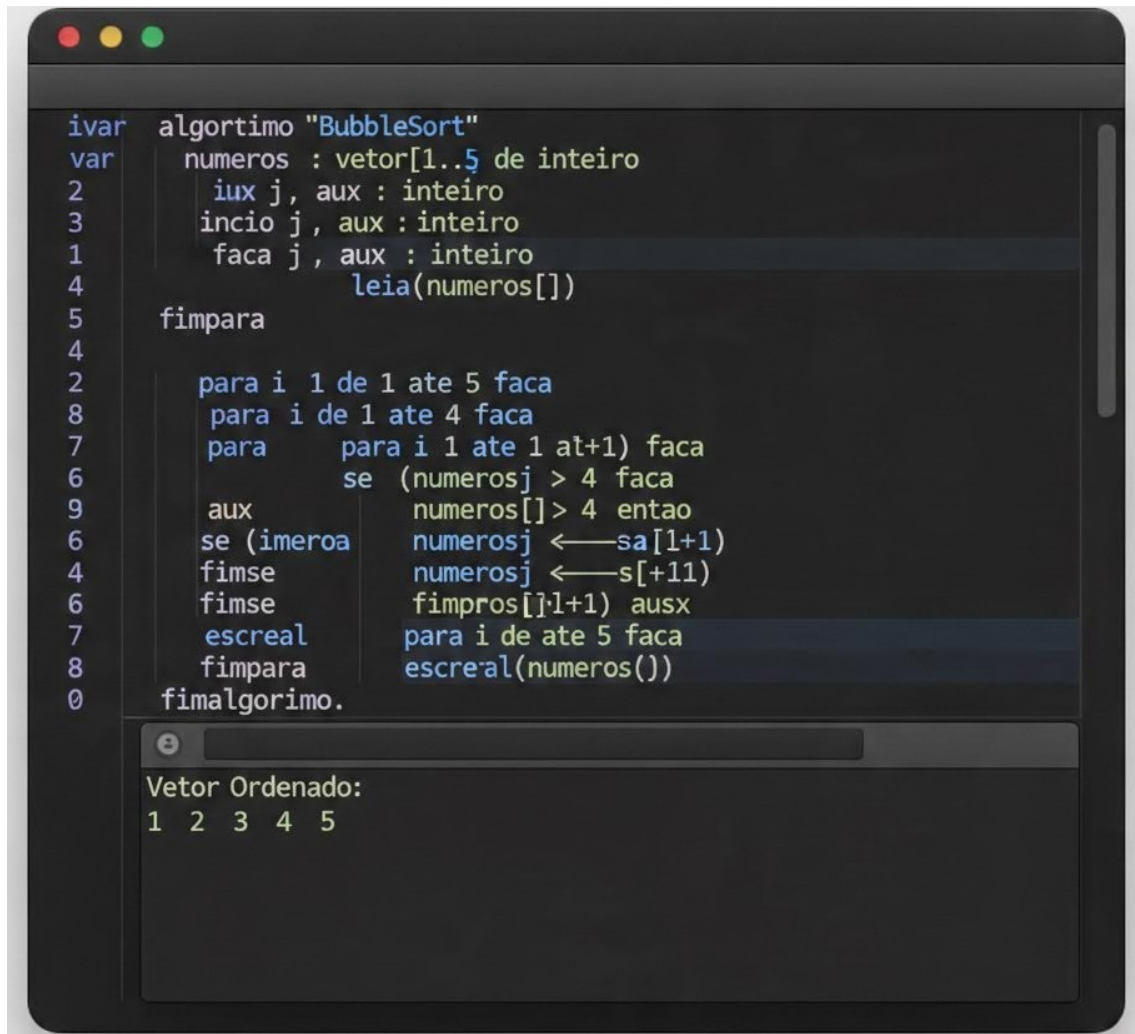
8 > 5

Troca:

5 8 10 6

O processo continua até que o vetor esteja totalmente ordenado.

9.10 Implementação do Bubble Sort



```
ivar algoritmo "BubbleSort"
var   numeros : vetor[1..5 de inteiro
2     iux j, aux : inteiro
3     incio j, aux : inteiro
1     faca j, aux : inteiro
4       leia(numeros[])
5   fimpara
4
2     para i 1 de 1 ate 5 faca
8       para i de 1 ate 4 faca
7         para i 1 ate 1 at+1) faca
6           se (numerosj > 4 faca
9             aux      numerosj > 4 entao
6             se (imeroa  numerosj <—sa[1+1)
4             fimse      numerosj <—s[+11)
6             fimse      fimpros[j+1) auxx
7             escreal    para i de ate 5 faca
8             fimpara    escreal(numeros())
0   finalgorimo.
```

Vetor Ordenado:
1 2 3 4 5

9.11 Como Funciona a Troca?

Suponha:

A = 10

B = 5

Objetivo:

A = 5

B = 10

Passo 1:

aux <- A

Resultado:

aux = 10

Passo 2:

A <- B

Resultado:

A = 5

Passo 3:

B <- aux

Resultado:

B = 10

Troca concluída.

9.12 Exemplo Completo

Vetor inicial:

8 3 6 1 9

Primeira passagem:

3 6 1 8 9

Segunda passagem:

3 1 6 8 9

Terceira passagem:

1 3 6 8 9

Resultado final:

1 3 6 8 9

9.13 Complexidade dos Algoritmos

Ao comparar algoritmos, utilizamos o conceito de complexidade.

A complexidade mede a quantidade de trabalho realizada pelo algoritmo.

Busca Sequencial

Percorre todos os elementos.

Representação:

$O(n)$

Bubble Sort

Realiza diversas comparações e trocas.

Representação:

$O(n^2)$

Embora não seja o algoritmo mais eficiente, é excelente para aprendizado.

9.14 Aplicações Reais

Busca:

Localizar clientes.

Pesquisar produtos.

Consultar alunos.

Ordenação:

Classificar notas.

Ordenar vendas.

Organizar relatórios.

Praticamente todo sistema utiliza busca e ordenação em algum momento.

9.15 Busca e Ordenação Trabalhando Juntas

Uma estratégia muito comum é:

1. Ordenar os dados.
2. Realizar buscas.

Exemplo:

Notas dos alunos.

→ Ordenar.

→ Localizar uma nota específica.

Esse processo é utilizado em sistemas corporativos, bancos e plataformas digitais.

Boas Práticas

- ✓ Utilize vetores adequadamente.
 - ✓ Escolha algoritmos compatíveis com o problema.
 - ✓ Faça testes de mesa.
 - ✓ Evite duplicação de código.
 - ✓ Utilize funções para organizar algoritmos.
 - ✓ Documente partes complexas.
-

Resumo do Capítulo

Neste capítulo aprendemos:

- O conceito de busca.
- Busca sequencial.
- Localização de elementos.
- Conceito de ordenação.
- Algoritmo Bubble Sort.
- Troca de valores.
- Complexidade algorítmica.
- Aplicações práticas.

Esses conceitos representam a base de diversos sistemas computacionais modernos e serão utilizados em praticamente todas as linguagens de programação.

Exercícios Propostos

1. Crie um algoritmo de busca sequencial para um vetor com 10 posições.
2. Informe a posição onde um número foi encontrado.
3. Conte quantas vezes um valor aparece em um vetor.
4. Desenvolva um algoritmo que encontre o maior valor de um vetor.
5. Desenvolva um algoritmo que encontre o menor valor de um vetor.
6. Implemente o Bubble Sort para ordenar 10 números.
7. Ordene um vetor em ordem crescente.
8. Ordene um vetor em ordem decrescente.
9. Faça o teste de mesa do Bubble Sort para:
5 3 1 4 2

10. Pesquise outros algoritmos de ordenação e compare-os com o Bubble Sort.
11. Crie uma função para realizar busca sequencial.
12. Desenvolva um sistema que cadastre notas, ordene os valores e permita buscar uma nota específica.
13. Explique a diferença entre busca e ordenação.
14. Explique o conceito de complexidade algorítmica.
15. Faça um relatório comparando a eficiência de um vetor ordenado e um vetor não ordenado.

CAPÍTULO 10 – BOAS PRÁTICAS E CLEAN CODE

10.1 Introdução

Aprender a programar não significa apenas criar algoritmos que funcionam.

Um algoritmo pode produzir o resultado correto e, ainda assim, ser considerado ruim do ponto de vista profissional.

Na área de desenvolvimento de software, um código deve possuir características que facilitem sua compreensão, manutenção e evolução.

Essas características fazem parte do conjunto de práticas conhecido como Clean Code, ou Código Limpo.

O objetivo deste capítulo é apresentar princípios fundamentais para a criação de algoritmos organizados, legíveis e profissionais.

10.2 O que é Clean Code?

O termo Clean Code foi popularizado pelo engenheiro de software Robert C. Martin, conhecido na comunidade de desenvolvimento como "Uncle Bob".

Código limpo é um código que:

- Funciona corretamente.
- É fácil de ler.
- É fácil de modificar.
- Possui organização lógica.
- Facilita a identificação de erros.

Um bom algoritmo deve ser compreendido por outros programadores e também pelo próprio autor após meses ou anos de sua criação.

10.3 A Importância da Legibilidade

Considere o seguinte algoritmo:

Exemplo Ruim

```
algoritmo "Teste"
```

```
var
```

```
a,b,c : real
```

```
inicio
```

```
leia(a)
```

```
leia(b)
```

```
c <- (a+b)/2
```

```
escreval(c)
```

```
fimalgoritmo
```

O programa funciona corretamente.

Porém, não é possível identificar facilmente o significado das variáveis.

Exemplo Melhorado

```
algoritmo "CalculoMedia"
```

```
var
```

```
nota1, nota2, media : real
```

```
inicio
```

```
leia(nota1)
```

```
leia(nota2)
```

```
media <- (nota1 + nota2) / 2
```

```
escreval(media)
```

```
finalgoritmo
```

O comportamento é exatamente o mesmo.

Entretanto, a leitura tornou-se muito mais simples.

10.4 Nomes Significativos

Os nomes utilizados em variáveis, funções e procedimentos devem representar claramente sua finalidade.

Evite:

a

b

x

y

z

Prefira:

media_aluno

idade_cliente

salario_funcionario

quantidade_produtos

Ao escolher nomes adequados, o código torna-se praticamente autoexplicativo.

10.5 Padronização

Um projeto deve seguir um padrão de nomenclatura.

Exemplo:

nome_aluno

media_final

quantidade_itens

Evite misturar estilos:

nomeAluno

MEDIA

QuantidadeItens

qtd

A consistência facilita a leitura e manutenção do código.

10.6 Comentários

Comentários são utilizados para explicar partes específicas do algoritmo.

Exemplo:

```
// Calcula a média final do aluno
```

```
media <- (nota1 + nota2) / 2
```

Contudo, comentários não devem substituir um código mal escrito.

Comentário Desnecessário

```
contador <- contador + 1
```

```
// Soma 1 ao contador
```

O comentário não acrescenta informação relevante.

Comentário Útil

```
// Regra institucional:
```

```
// Alunos com média entre 5 e 6,9 entram em recuperação.
```

```
se (media >= 5) e (media < 7) entao
```

Nesse caso o comentário ajuda a compreender uma regra de negócio.

10.7 Indentação

Indentação é o alinhamento visual das estruturas do algoritmo.

Ela melhora significativamente a legibilidade.

Exemplo inadequado:

```
se (idade >= 18) entao
```

```
escreval("Maior")
```

```
senao
escreval("Menor")
fimse
```

Exemplo correto:

```
se (idade >= 18) entao
```

```
    escreval("Maior")
```

```
senao
```

```
    escreval("Menor")
```

```
fimse
```

A estrutura torna-se muito mais clara.

10.8 Evitando Repetição de Código

A repetição excessiva de código aumenta a manutenção do sistema.

Exemplo inadequado:

```
escreval("Sistema Escolar")
```

```
escreval("Sistema Escolar")
```

```
escreval("Sistema Escolar")
```

Melhor solução:

```
procedimento Cabecalho()
```

```
inicio
```

```
    escreval("Sistema Escolar")
```

```
fimprocedimento
```

Depois:

Cabecalho()

Cabecalho()

Cabecalho()

Essa técnica reduz erros e facilita alterações futuras.

10.9 Funções com Responsabilidade Única

Uma função deve realizar apenas uma tarefa.

Exemplo inadequado:

Função calcula média,
lê dados,
imprime resultados
e grava relatório.

Exemplo adequado:

Funcao CalcularMedia()

Procedimento ExibirResultado()

Procedimento GerarResultado()

Cada módulo possui apenas uma responsabilidade.

10.10 Variáveis Globais

Variáveis globais podem ser acessadas por todo o programa.

Exemplo:

var

contador_global : inteiro

Embora sejam úteis em algumas situações, seu uso excessivo pode dificultar a manutenção.

Sempre que possível, utilize parâmetros e variáveis locais.

10.11 Testes de Mesa

Antes de executar um algoritmo, é recomendável realizar testes de mesa.

Exemplo:

```
x <- 10
```

```
y <- 20
```

```
soma <- x + y
```

Simulação:

Variável	Valor
----------	-------

x	10
---	----

y	20
---	----

soma	30
------	----

Essa prática permite identificar erros lógicos antes da execução.

10.12 Tratamento de Entradas

Uma boa prática consiste em validar informações fornecidas pelo usuário.

Exemplo:

```
se (idade < 0) entao
```

```
    escreval("Idade invalida")
```

```
fimse
```

Essa validação evita resultados incorretos.

10.13 Complexidade Desnecessária

Um erro comum é criar soluções excessivamente complexas para problemas simples.

Exemplo:

Problema:

Somar dois números.

Solução correta:

```
resultado <- numero1 + numero2
```

Evite criar estruturas desnecessárias que dificultem a compreensão.

10.14 Código Espaguete

Código espaguete é o nome dado a algoritmos excessivamente desorganizados.

Características:

- Muitas estruturas aninhadas.
- Repetição excessiva.
- Baixa legibilidade.
- Dificuldade de manutenção.

Exemplo:

```
se (condicao1) entao
  se (condicao2) entao
    se (condicao3) entao
      se (condicao4) entao
        escreval("Executar")
      fimse
    fimse
  fimse
fimse
```

Sempre que possível, simplifique estruturas complexas.

10.15 Organização de Algoritmos

Uma organização recomendada é:

Algoritmo

Constantes

Variáveis

Funções

Procedimentos

Bloco Principal

Essa estrutura facilita a navegação pelo código.

10.16 Desenvolvimento Profissional

Na indústria de software, grande parte do trabalho consiste em manter e evoluir sistemas já existentes.

Por esse motivo, escrever código limpo é tão importante quanto escrever código funcional.

Um algoritmo organizado:

- ✓ Reduz custos.
 - ✓ Facilita correções.
 - ✓ Melhora a produtividade.
 - ✓ Reduz a ocorrência de erros.
 - ✓ Facilita o trabalho em equipe.
-

Resumo do Capítulo

Neste capítulo aprendemos:

- Conceito de Clean Code.
- Importância da legibilidade.
- Nomes significativos.
- Comentários adequados.
- Indentação.
- Modularização.
- Reutilização de código.
- Responsabilidade única.
- Teste de mesa.
- Boas práticas profissionais.

Esses princípios representam a transição entre um programador iniciante e um desenvolvedor capaz de construir algoritmos organizados e sustentáveis.

Exercícios Propostos

1. Reescreva um algoritmo utilizando nomes de variáveis mais significativos.
2. Identifique comentários desnecessários em um código.
3. Corrija problemas de indentação em um algoritmo.

4. Transforme trechos repetidos em procedimentos.
5. Crie uma função com responsabilidade única.
6. Faça um teste de mesa de um algoritmo de sua escolha.
7. Reorganize um algoritmo seguindo a estrutura apresentada neste capítulo.
8. Explique com suas palavras o conceito de Clean Code.
9. Cite três vantagens da modularização.
10. Explique a diferença entre um código funcional e um código limpo.
11. Identifique problemas em um algoritmo com excesso de estruturas aninhadas.
12. Reescreva um algoritmo utilizando boas práticas de programação.
13. Explique o que é código espaguete.
14. Desenvolva um algoritmo simples aplicando todas as boas práticas estudadas.
15. Faça uma análise crítica de um algoritmo desenvolvido anteriormente neste livro, identificando pontos que poderiam ser melhorados.

CONCLUSÃO

Ao longo deste livro, percorremos uma jornada que começou com conceitos simples, como algoritmos e variáveis, e avançou gradualmente até estruturas de repetição, vetores, matrizes, funções, procedimentos e algoritmos clássicos. Mais do que aprender comandos de Portugol, o objetivo foi desenvolver uma forma diferente de pensar e resolver problemas.

A programação costuma ser vista por iniciantes como algo extremamente complexo, reservado apenas para pessoas com grande conhecimento matemático ou técnico. Na prática, porém, programar é um processo de aprendizado contínuo, baseado principalmente em lógica, persistência e experimentação. Nenhum programador experiente começou sabendo tudo. Todos passaram por momentos de dúvida, erros constantes e longos períodos tentando entender por que um algoritmo aparentemente correto simplesmente não funcionava.

Durante os estudos, é comum encontrar dificuldades que nem sempre possuem uma solução imediata. Alguns erros de lógica podem consumir horas ou até dias de análise. Em outras situações, o algoritmo executa sem apresentar erros, mas produz resultados incorretos devido a um detalhe que passou despercebido. Também existem problemas mais complexos que exigem conhecimentos além do conteúdo introdutório apresentado neste livro, envolvendo estruturas de dados avançadas, bancos de dados, redes de computadores e engenharia de software.

Outro aspecto importante é compreender que a programação nem sempre segue um caminho linear. Muitas vezes uma solução aparentemente perfeita precisa ser refeita diversas vezes até atingir um resultado satisfatório. Essa realidade faz parte do processo de desenvolvimento e não deve ser encarada como fracasso. Pelo contrário, os erros e dificuldades normalmente representam as maiores oportunidades de aprendizado para qualquer programador.

Apesar dos desafios, existe uma satisfação única em criar algo próprio e vê-lo funcionando. Poucas experiências são tão recompensadoras quanto transformar uma ideia em um algoritmo, executar o programa e observar que ele realiza exatamente aquilo que foi planejado. Seja um simples cálculo de média, um sistema de cadastro ou um projeto mais elaborado, cada programa concluído representa a materialização de conhecimento, criatividade e esforço.

À medida que novas habilidades são adquiridas, surgem possibilidades cada vez maiores. O estudante que hoje desenvolve algoritmos em Portugol poderá, futuramente, criar aplicações web, sistemas empresariais, jogos, aplicativos móveis, ferramentas de automação e até participar do desenvolvimento de tecnologias que impactam milhões de pessoas.

Este livro não representa o fim da jornada, mas apenas o primeiro passo. A programação é uma área vasta, dinâmica e em constante evolução. Sempre haverá novos desafios, novas linguagens e novas tecnologias para aprender. O mais importante é manter a curiosidade, praticar regularmente e não desistir diante das dificuldades.

Se existe uma lição central em toda a lógica de programação, talvez seja esta: grandes sistemas são construídos a partir de pequenos passos. Da mesma forma, grandes programadores surgem da soma de inúmeros algoritmos simples, erros corrigidos, problemas resolvidos e projetos concluídos.

Que este material tenha servido como base para o desenvolvimento do raciocínio lógico e como incentivo para continuar explorando o universo da programação. O próximo projeto, a próxima ideia e a próxima solução agora dependem apenas da sua criatividade e da sua dedicação.

Boa sorte em sua jornada como programador.

Referencias

FORBELLONE, André Luiz Villar; EBERSPÄCHER, Henri Frederico.
Lógica de Programação. 3. ed. São Paulo: Pearson, 2005.

MANZANO, José Augusto N. G.
Algoritmos: Lógica para Desenvolvimento de Programação.
29. ed. São Paulo: Érica, 2019.

ASCENCIO, Ana Fernanda Gomes;
CAMPOS, Edilene Aparecida Veneruchi.
Fundamentos da Programação de Computadores.
3. ed. São Paulo: Pearson, 2012.

MARTIN, Robert C.
Código Limpo.
Rio de Janeiro: Alta Books, 2009.